Solar-Panel Sun Tracking Device

Ana Luo[#1], Cyrus Lowden[#2], Luis Albos[#3], Elise Esquitin[#4], Rebecca Duran[#5]

[#]*Department of Engineering Technology and Industrial Distribution, Texas A&M University*
Fermier Hall, 106 Ross St., College Station, TX 77843-3367, United States

[1]ana.luo1258@tamu.edu

[2]heckie_m0m0@tamu.edu

[3]luis_albos@tamu.edu

[4]evesquire@tamu.edu

[5]rebecca479@tamu.edu

***Abstract*— The project is a solar panel that tilts on an axis to follow the sun as it moves perpendicular to the sky. This allows for a solar panel not to be limited to only obtaining the maximum amount of sunlight at peak hours throughout the day. LR sensors on each corner of the device can determine the amount of sunlight being obtained on each side, and the panel will tilt accordingly so that the maximum amount of sunlight is being taken in. This will be done by comparing the signals from the two sensors; the system will tilt towards the sensor with the greater signal. A second-order PD controller is used to control the system, with a settling time of 8s, a steady state error of 3 degrees, and an overshoot of <5%.**

***Keywords*— Solar Tracker, Arduino, PD Control**

I. Introduction

This project aims to make use of a solar panel, four photoelectric sensors, and a motor with encoder to angle the solar panel along one degree of motion. This solar panel will angle itself towards the brightest light determined by the photoelectric sensors, improving on stationary solar panel functionality. By moving the solar panel, it will be able to take in light outside of peak hours. This report covers the final result of the project, including the z-domain implementation, construction, and final validation.

System Architecture & Physical Implementation

I. Mechanical and Electrical Assembly

The solar tracking system consists of a solar panel, four photoelectric sensors, a TS25GA DC motor, an L298D H-bridge motor driver, an Arduino Mega 2560 R3 microcontroller, and a potentiometer for control adjustment. These components work together to form a single-axis tracking mechanism that allows the solar panel to rotate toward the direction of maximum light intensity.

The four photoelectric sensors serve as the primary sensing elements of the system. Their purpose is to detect variations in incident light intensity from different directions. Based on the sensor readings, the controller determines the direction of the brightest light source and generates the appropriate control action. The Arduino Mega 2560 R3 functions as the central processing unit of the system, receiving the sensor inputs, executing the control logic, and issuing output commands to the motor driver.

To physically reposition the solar panel, a TS25GA DC motor is used as the actuator. Since the motor cannot be driven directly by the microcontroller, an L298D H-bridge is included to provide the required power interface and bidirectional motor control. This allows the panel to rotate in either direction depending on the light conditions.

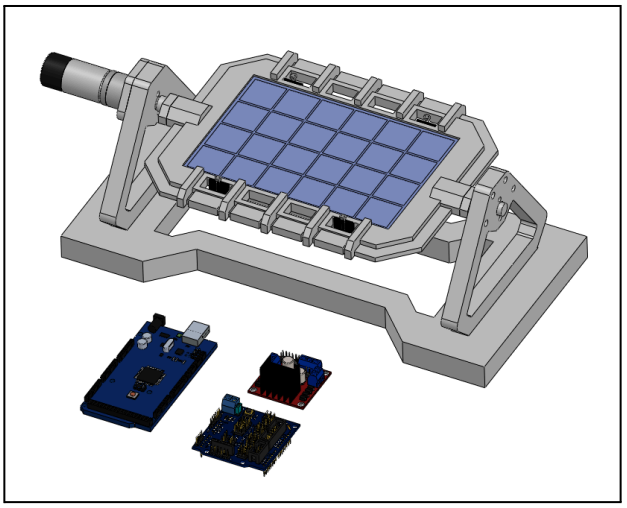


Fig. 1. Final Solar Tracker CAD Assembly

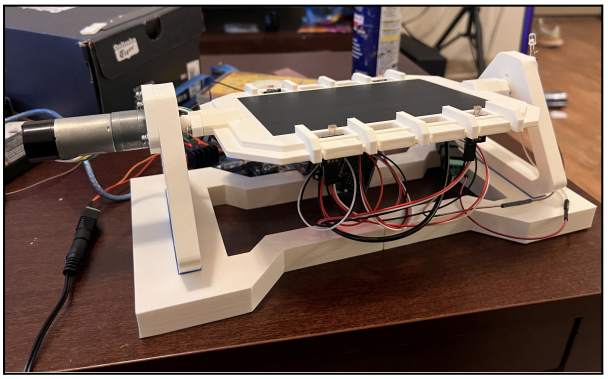


Fig. 2. Fully manufactured Solar Tracker

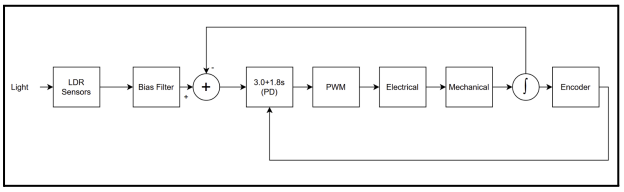


Fig. 3. Final Block Diagram

II. Specification of feedback loop components

The components used to make the feedback loop are the KY-018 photoresistors and the encoder of the TS25GA DC motor. With the voltage of each photoresistor after converting from ADC, a relationship between light intensity and the value is established. With this connection a feedback loop is able to be made depending on the intensity of the light which will be important for the algorithm implemented to move to the light source. The encoder is necessary for the physical control of the mechanical system so the solar panel will respond to the appropriate position.

III. Transfer functions

To derive the equation needed to model the system plant, the model is separated into two separate components. The mechanical portion is modeled as the motor and the attached solar panel load which will only be moving along a single-axis. The mechanical differential equation acts as the main plant for the solar panel system The modeled equation for the mechanical system is shown below.

$$k_t i(t) = J(\frac{d\omega(t)}{dt}) + b * \omega(t) \qquad (1)$$

where K_T is the motor torque constant, i(t) is current, J is rotational inertia, w(t) is angular velocity, and b is viscous damping. The electrical side of the system is modeled with the voltage and current.

$$V(t) = L(\frac{di(t)}{dt}) + Ri(t) + ke * \omega(t) \qquad (2)$$

Where $V_M(T)$ is the armature voltage, L is inductance, R is resistance, and ke is the back emf constant.For the overall system equation, (1) and (2) are combined as seen below.

$$J\theta''(t) + (b + \frac{K_t K_e}{R})\vartheta'(t) = (\frac{K_t}{R})V_m(t) \qquad (3)$$

Where θ is the angular position of the solar panel.

The input of the Physical plant, is the effective motor voltage, V_M;

$$u(t) = V_m(t) \qquad (4)$$

The output would be the angular position of the solar panel;

$$y(t) = \theta(t) \qquad (5)$$

The model consists of four sensors with walls between them so there is no cross-illumination between them. As this is a single-axis, a horizontal tracking error is defined by:

$$e(t) = \frac{(V_{TR}+V_{BR})-(V_{TL}+V_{BL})}{(V_{TR}+V_{BR})+(V_{TL}+V_{BL})+\varepsilon} \qquad (6)$$

The four sensor voltages are defined by their locations: top left (TL), top right (TR), bottom right (BR), and bottom left (BL). The small constant to avoid division by zero is defined by the ε. This is to ensure there is more continuous control and less sensitivity to overall brightness changes.

The selected actuator is a TS-25GA370H brushed DC gearmotor with encoder, specifically the 12 V, 600 RPM version from the user-provided specification table. That table also shows a reduction ratio of 1:10 and 120 pulses per output-shaft revolution. This gives an angular resolution of $360^\circ/408 = 3^\circ$/ pulse. This encoder resolution is sufficient for measuring panel position and estimating angular velocity for the derivative portion of the controller.

The model coefficients are:

$$J = Equivalent\ Rotational\ Inertia$$

$$B = b + \frac{K_t K_e}{R}$$

$$K = \frac{K_t}{R}$$

R, K_T, and K_E are motor parameters, while J depends on the conceptual and physical layout of the rotating structure. The motor is driven through an L298 full-bridge driver, which is designed to accept standard TTL logic levels and drive DC motors. Because the L298 introduces internal voltage drop, the motor terminal voltage is not exactly equal to the supply voltage times PWM duty cycle. In implementation, the effective plant input is better represented as an average voltage.

$$V_m(t) \approx \alpha d(t) V_s - V_{drop} \quad (7)$$

D(t) is PWM duty cycle, V_S is the supply voltage, α is the scaling factor, and V_{DROP} represents H-bridge losses.

The solar panel mass is taken from the data sheet as 80 grams.

$$m_{panel} = 0.08\,kg$$

The panel inertia is approximated as a thin rectangular plate rotating about an axis through its centroid and perpendicular to its face.

$$J_{panel} \approx \frac{1}{12} m(a^2 + b^2)$$

A and B are panel length and width. The total equivalent inertia is TH estimated as:

$$J \approx J_{motor,reflected} + J_{mount} + J_{panel}$$

Utilizing the differential equation calculated prior, the transfer function of the system can be obtained. A laplace transform can be performed on the differential equation to get the output of equation 12. By isolating the variables, the plant transfer function can be determined to be equation 13.

$$Js^2\Theta(s) + Bs\Theta(s) = KVm(s) \quad (8)$$

$$\frac{\Theta(s)}{Vm(s)} = \frac{K}{Js^2+Bs} \quad (9)$$

For a standard closed-loop transfer function, the equation is seen as follows:

$$C(s)G(s) = \frac{C(s)G(s)}{1+C(s)G(s)} \quad (10)$$

$$C(s)=K_P+K_D*s \quad (11)$$

By using equation (10), the closed-loop transfer function is found.

$$C(s)G(s) = \frac{(\frac{Kd*Kt}{R})s+(\frac{Kp*Kt}{R})}{Js^2+(B+(\frac{Kd*Kt}{R}))s+(\frac{Kp*Kt}{R})} \quad (12)$$

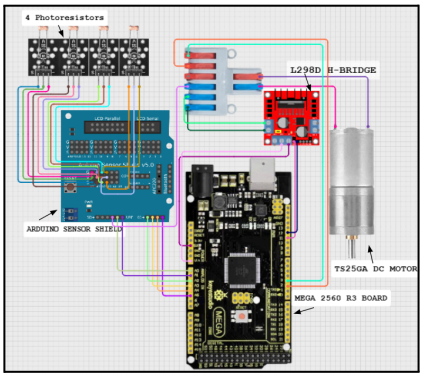


Fig. 4. Finalized Circuit Interface Schematic

Figure 4 above shows the finalized schematic. The photoresisters are connected to the analog pins A0-A4 on the expansion board. The analog pins from the Arduino Mega are connected to the analog input pins from 0-4. The board receives 5V power from a charging bank. The Arduino is connected to the H-pins from pins 11 and 12 for the encoders and 2 and 3 for the digital input pins. The H-bridge receives 12V from a wall adapter power supply.

Table 1. Finalized Component BOM

Component	Estimated Price
MEGA 2560 R3 Board	$50
L298D H-Bridge	$0 (obtained from lab)
KY-018 photoresistor 4x	$5
Solar Panel	$12
Wiring	$0 (obtained from lab)
ts25ga DC motor	$0 (obtained from lab)
3D Printed Mount	$0 (Obtained from Luis)
Mount Screws	$12
18650 battery Case	$3
USB Breakout Board	$3
x1 Red LED	$0 (Already Had)

18650 batteries	$24
Potentiometer	$0 (Obtained from lab)

The finalized hardware bill of materials for the proposed solar tracking system is summarized in Table 1.

Some of the required components were obtained from existing laboratory resources, which significantly reduces the overall implementation cost. These lab-supplied components include the

Arduino Mega 2560 R3 board, L298D H-bridge, wiring, TS25GA DC motor, and potentiometer, each contributing no additional direct expense to the project budget.

Controller Design & Tuning

I. Define Target metrics

The settling time was determined to be around 8 seconds; the sun is slow-moving, and it was determined that a faster settling time was unnecessary. A low overshoot of under 10% was desired; a high overshoot would wear down the motor and not be useful for a slow-moving light source. Unfortunately, our overshoot ended up much higher. Because a PD controller is being used, a steady state error of 0 is not achievable; the steady state error was determined to be 3 degrees, in comparison to our proposed error of less than 5 degrees.

II. Chosen gains

PD was chosen for this project since, with the four sensors used in the design, the steady state will naturally be low. This is because the four sensors average out the light data, and the solar panel moves to align to a position where each average sensor reading is almost the same.

The controller gains are found by comparing the closed-loop denominator with the desired characteristic equation.

$$s^2 + 2\zeta\omega n s + \omega n^2 = 0 \qquad (13)$$

$$s^2 + (B + \frac{Kd \cdot Kt/R}{J})\, s + (\frac{Kp \cdot Kt/R}{J}) \qquad (14)$$

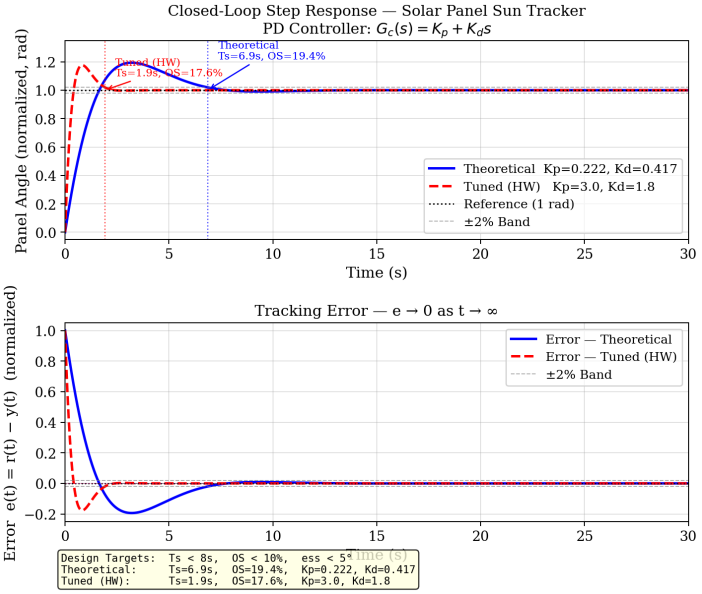
From the equations above Kp, and Kd equations are solved for

$$Kp = (\omega n^2)(J) * \frac{R}{Kt} \qquad (15)$$

$$Kd = (2\zeta\omega n J - B) * \frac{R}{Kt} \qquad (16)$$

We ended up with a calculated KP value of 0.22 and a Kd value of 0.417; however, these did not result in the desired behavior of the device. The values were adjusted manually in the code until a Kp value of 3.0 and a Kd value of 1.8 were determined.

III. Closed Loop Step Response



IV. Fig. 5. Closed Loop Step Response Graph

Figure 5 shows the closed-loop step response for both the theoretical gains and the hardware-tuned gains. The initial kp was 0.22 and Kd was 0.417 but final values were determined to be 3.0 and 1.8 respectively. Both gain sets exceed the 5% overshoot target. This is expected as the closed-loop transfer function contains a zero in the numerator from the derivative term.There is a deadzone in the motor which makes the response nonlinear.

Embedded System Implementation

I. Tustin's Bilinear Transformation

To convert the equation from the s-domain to the z-domain, Tustin's transformation formula is applied, substituting for s as shown in (5).

$$s = (\frac{2}{T_s})(\frac{z-1}{z+1}) \qquad (17)$$

By doing the substitution on the controller $Gc(s)$, the solution is found as shown below after simplification.

$$H(z) = \frac{C(z)}{E(z)} = \frac{(Kp + 2Kd/Ts)\cdot z + (Kp - 2Kd/Ts)}{z+1} \quad (18)$$

II. Difference Equation Derivation

To get the difference equation, C(s) is isolated using *(18)* to solve. The equation is then shifted by one sample delay then it is converted to the time domain. This gives the solution seen below.

$$u_k = -u_{k-1} + (Kp + \frac{2Kd}{Ts})e_k + (Kp - \frac{2Kd}{Ts})e_{k-1} \quad (19)$$

III. Sampling Frequency Justification

The sampling frequency for the system will be fairly slow, considering the real-life application will be tracking the sun, which moves very slowly. Using the equations below to get an estimation, the natural frequency is around 0.114 Hz.

$$Ts = \frac{4}{\zeta\omega n} \quad (20)$$

$$Fn = \frac{\omega n}{2\pi} \quad (21)$$

Where Ts is at 8 rad/second and ζ is 0.7.

The sun rotates at about 15° per hour relative to the Earth's surface. Considering there are 12 to 13 hours of daylight on average, the panel has that many hours to travel 180° in total, which does not require a high frequency.

Software Implementation (FreeRTOS)

I. Task Architecture

The final embedded software used FreeRTOS to separate the solar tracker into real-time tasks for sensing, control, actuation, and telemetry. This improved modularity and made the software easier to manage and verify. The system used four tasks: Sensor, Control, Driver, and Telemetry. The Control task had the highest priority because it directly affected closed-loop behavior. The Sensor and Driver tasks used intermediate priority, while the Telemetry task used the lowest priority because serial output was only used for monitoring and debugging. The final task priorities were Sensor = 2, Control = 3, Driver = 2, and Telemetry = 1.

The Sensor task read the four LDR channels, converted the raw ADC values into brightness scores, and classified the light position as top, center, or bottom. These positions corresponded to commanded panel angles of +45°, 0°, and -45°, respectively. A confirmation delay was used so the detected light sector had to remain stable before the target angle changed, reducing false switching from noise or temporary lighting changes. When the command changed directly between +45° and -45°, the software first moved the panel to 0° so the mechanism passed through center. The relationship between these software subsystems is shown in Figure 6.

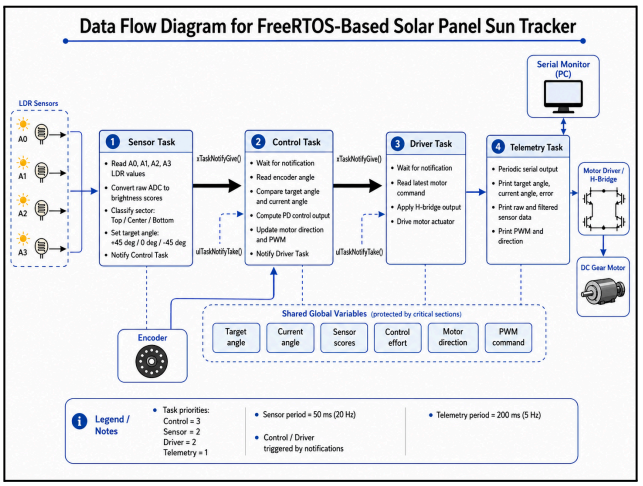


Fig. 6: Data Flow Diagram

The Control task used the latest encoder angle and target angle to calculate the motor command. The Driver task applied the direction and PWM output to the motor driver. The Telemetry task sent diagnostic data such as sensor readings, light metrics, angle, target, control effort, and actuator command. This task separation kept sensing, control, actuation, and monitoring organized within the real-time sequence.

The final implementation used shared global variables for data transfer and FreeRTOS task notifications for synchronization. Shared variables stored the latest sensor readings, brightness scores, angle, target angle, control effort, direction command, and PWM value. Since only the most recent system state was needed, queues were not required. Critical sections using taskENTER_CRITICAL() and taskEXIT_CRITICAL() protected grouped reads and writes between tasks.

Task notifications controlled the main sequence. After each sensing cycle, the Sensor task released the Control task using xTaskNotifyGive(). The Control task waited with ulTaskNotifyTake() until new sensor data was available, then calculated the motor command. It then released the Driver task, which waited until the updated command was ready before applying it to the motor. This created a deterministic sensing → control → actuation sequence.

The Telemetry task was not included in this notification chain. It ran independently at a lower priority and only

read shared variables for reporting, preventing serial output from interfering with the real-time control sequence.

II. Task Timing and Deadline Performance

Task timing was based on the solar tracker's physical speed and each task's role. The Sensor task ran every 50 ms, or 20 Hz, which was sufficient because light direction changes slowly. The Control and Driver tasks were notification-driven, so they also updated once per sensor cycle. The Telemetry task ran every 200 ms, or 5 Hz, which was enough for serial monitoring without overloading the system.

The tasks were expected to meet their deadlines because the sensing, control, and driver calculations were lightweight. The notification chain ensured that Control only ran after new sensor data was available, and Driver only ran after a new control command was produced. This made the sequence more deterministic than simple polling. Telemetry was the main possible source of delay, but it ran at the lowest priority and did not affect the control chain.

The final implementation also supported real-time telemetry of raw and processed sensor signals. This allowed the team to verify that filtering reduced sensor noise, as shown in Figure 7.

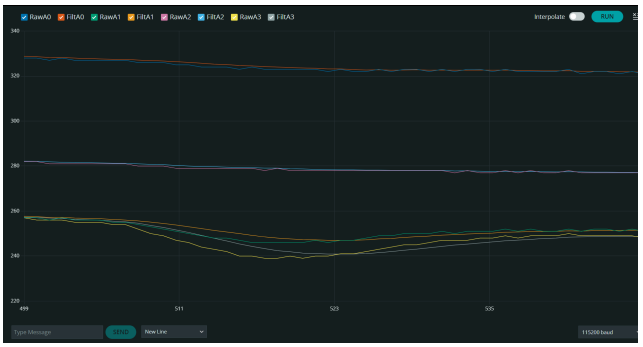


Fig. 7. Raw and Filtered Data Plot

Experimental Results & Performance Analysis

I. Final Closed-Loop Performance

The final system was evaluated by moving a flashlight relative to the photoresistor array and observing the panel response. The light was directed straight above the sensor array, then shifted to the top quadrant, then to the bottom quadrant, and finally returned to the straight-above position. Based on the detected light sector, the controller commanded the panel to the corresponding target angle of 0°, $+45^\circ$, or -45°.

During testing, the panel successfully moved to the appropriate target angle for each lighting condition, demonstrating that the sensing, sector classification, and feedback control stages operated together as intended.

The final closed-loop behavior is summarized in Figure 8, which shows the reference, output, error, and control effort over the full test sequence. The output followed the commanded angle changes with visible transient behavior, while the error decreased as the panel approached each target.

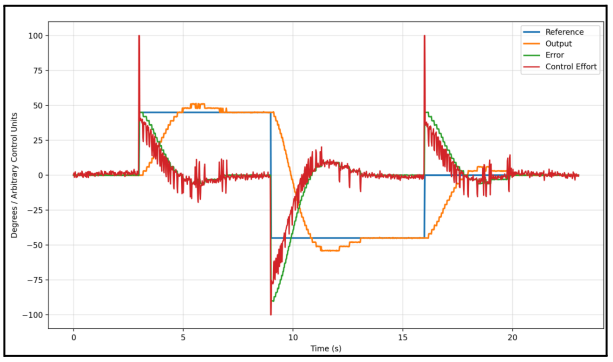


Fig. 8. Final Test Performance Plot

II. Error Response

The tracking error decreased as the panel moved toward the commanded angle, indicating that the closed-loop system operated properly. In the final tuned implementation, the steady-state error was approximately 3°. This value was also consistent with the encoder resolution obtained from the motor datasheet. Since the angular resolution is determined by $360^\circ/\text{PPR}$, where the PPR is 120.

The final steady-state error was not only observed experimentally, but also matched the practical resolution limit of the feedback measurement. In addition, because the controller was PD rather than PID, a small nonzero steady-state error was expected under real loading conditions.

III. Sensor Filtering

Raw LDR measurements contained noticeable fluctuation from ambient lighting, sensor mismatch, and electrical noise. During demonstration, both direct sunlight and classroom ceiling lights interfered with the sensor readings. To improve robustness, the four sensor readings were grouped into combined top, bottom, left, and right light sums rather than relying

on a single sensor reading alone. For the single-axis tracker, the primary decision variable was the balance between the top and bottom sensor groups.

A normalized vertical bias was then used to classify the light sector. This allowed the controller to respond to the relative difference between top and bottom illumination instead of the absolute brightness level alone, making the system less sensitive to uniform changes in ambient light. A center threshold was applied so that when the top and bottom sums were sufficiently close, the system treated the light distribution as centered and commanded $0°$.

IV. Simulated and Real Response Comparison

A comparison between the modeled response and the real hardware response is shown in Figure 9. Both responses followed the same general trend, since each reduced error and moved the panel toward the commanded reference. However, the real hardware response was less smooth and more irregular than the simulated response.

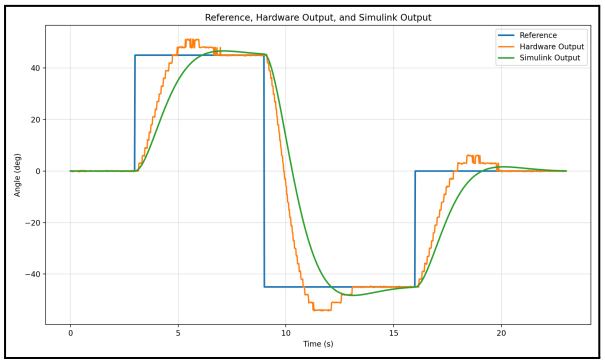


Fig 9. Simulated vs. Real Telemetry Data

Several physical factors caused this difference. First, the 600 RPM motor was too fast for the slow single-axis actuation required by this project, which made the real motion more aggressive than desired. Second, the encoder pulse-per-revolution resolution was not precise enough to provide smooth position feedback, causing the measured angle to change in coarse increments. Third, the physical system included backlash, structural compliance, and varying friction that were not fully represented in the model.

An additional source of error came from the wiring attached to the moving solar panel assembly. The wires connecting the solar panel LED and the photoresistors back to the shield and motor driver imposed a downward mechanical load on the structure. In practice, this wire weight and tension tended to pull the panel back toward the $0°$ position, especially near the ends of travel. This effect opposed the commanded motion and was not included in the simulation model, which contributed to the difference between the real and simulated responses. Finally, ambient light from both the sun and classroom ceiling lights interfered with the sensor readings during demonstration, introducing disturbances that were not present in the simulation.

V. Performance Differences

Compared to simulation, the hardware exhibited greater overshoot, less smooth settling, and noisier feedback. These differences were primarily due to unmodeled mechanical backlash, actuator nonlinearity, limited encoder resolution, and environmental lighting interference. Although the real system did not match the ideal simulated response, it still demonstrated the intended single-axis solar tracking behavior and validated the overall control approach.

Discussion

The major project tasks were ultimately completed, including design, fabrication, assembly, software integration, controller tuning, and final demonstration. As shown in Table 8, the planned milestones of report completion, component arrival, construction, debugging, and demo preparation were all reached. However, the timeline shifted in practice because hardware redesign, repeated fabrication, and debugging required more time than originally expected.

Table 2. Project Timeline

Week	Tasks
3/29	4/3 - Complete progress report
4/5	4/6 - Components arrive
4/12	Construct, Test, Debug Structure
4/19	Finalize debugging, Demo

A major showstopper was discovered when the initial finalized design placed the 300g solar panel approximately 10 cm above the motor. This created an estimated torque demand of about 3 $kg \cdot cm$, which exceeded the motor stall torque of 1 $kg \cdot cm$ and was far above its nominal operating torque of 0.25 $kg \cdot cm$. This made the original mechanical design infeasible, so the structure had to be redesigned only a few days before the final demonstration. The redesigned structure reduced the effective load on the motor and allowed

the mechanism to function, although actuator limitations still remained.

Additional delays occurred during manufacturing and assembly. Much of the final fabrication was completed the night before the demo using two 3D printers running simultaneously. Because of the compressed schedule, some parts were removed while still warm, which caused warping and forced reprints. There were also repeated tolerancing issues in threaded regions, screw interfaces, and the motor linkage because several early part designs had effectively zero tolerance. This led to multiple redesigns and reprints before acceptable fitment was achieved. These manufacturing problems significantly compressed the available debugging time.

Another important limitation was the selected motor itself. The available 600 RPM variant used only a 1:10 gear reduction ratio, which made the output much faster than desired for a slow single-axis solar tracker. Although this issue is discussed further in the performance analysis, it is important to note here because it directly affected integration, tuning, and debugging. The motor's speed and low torque margin made smooth low-speed positioning more difficult and increased the time required to stabilize the final hardware.

The debugging phase was also affected by encoder issues and general hardware non-idealities. Considerable time was spent obtaining encoder readings that were reliable enough for closed-loop control. Even after the encoder and control logic were functioning, the team still had to recalibrate and retune the system under the actual demonstration lighting conditions, since the room used during development was not as brightly lit as the classroom. This reduced the amount of time available for final refinement.

The final controller choice was PD. This controller was retained because it provided a practical balance between response speed and damping while remaining simple to implement on the embedded platform. The derivative term helped reduce oscillation and improve settling compared to a purely proportional approach, while the overall structure remained less complex than introducing additional integral action. Although the PD controller did not remove all real-world effects, it was sufficient to achieve a functional demonstration after hardware tuning.

Overall, the final system reached a working demonstration state, but only after significant redesign and debugging effort. The main factors that affected the schedule and final performance were mechanical overloading in the original design, repeated fabrication issues, limited encoder fidelity, and the use of a motor that was too fast for the intended actuation. If the project were repeated, a more appropriate motor would be selected earlier, realistic tolerances would be included from the start, and additional fabrication and testing time would be reserved before the final demo.

Conclusions

In conclusion, the solar panel sun tracking device successfully demonstrated the intended single-axis tracking behavior. The system detected changes in light position and moved the panel to the corresponding target angles of $+45^\circ$, 0°, and -45°. Although the final hardware response was somewhat clunky, the overall design metrics were met sufficiently to validate the control strategy and embedded implementation.

The main limitations of the final prototype were the delayed manufacturing schedule, the weak and overly fast motor, and the reduced time available for refinement. Starting fabrication earlier would likely have exposed the motor limitations sooner and reduced the amount of redesign required near the demonstration date. For future work, a stepper motor with encoder feedback would be a more suitable actuator because it would provide better low-speed positioning and improved control precision for this application.

Overall, the project achieved its primary objective of demonstrating a functioning closed-loop solar tracking system. The final contribution distribution for each team member is summarized in Table 3.

Table 3. Statement of Contribution

Name	Contribution %
Ana Luo	120%
Cyrus Lowden	120%
Luis Albos	120%
Elise Esquitin	70%
Rebecca Duran	70%
Total	500%

Acknowledgment

This team wishes to acknowledge Meghana Chintalapati for furthering the team's understanding of the laboratory and project assignments.

References

[1] Arduino, “Arduino Mega 2560 Rev3 Datasheet,” Arduino Documentation, Feb. 2026. [Online]. Available: https://docs.arduino.cc/resources/datasheets/A000067-datasheet.pdf

[2] Senba Sensing Technology, “GL5528 LDR Photoresistor Datasheet,” SparkFun-hosted datasheet. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/LightImaging/SEN-09088.pdf

[3] STMicroelectronics, “L298 Dual Full-Bridge Driver,” STMicroelectronics. [Online]. Available: https://www.st.com/en/motor-drivers/l298.html

[4] [Motor Manufacturer], “TS-25GA370H DC Gearmotor with Encoder Datasheet,” [Online]. Available: https://grabcad.com/library/geared-dc-motor-with-encoder-ts-25ga370h-1

[5] MathWorks, “Simulink Documentation,” MathWorks. [Online]. Available: https://www.mathworks.com/help/simulink/index.html

[6] MathWorks, “Simulink - Simulation and Model-Based Design,” MathWorks. [Online]. Available: https://www.mathworks.com/products/simulink.html

[7] FreeRTOS, “RTOS Task Notifications,” FreeRTOS Documentation. [Online]. Available: https://freertos.org/Documentation/02-Kernel/04-API-references/05-Direct-to-task-notifications/00-RTOS-task-notifications

[8] FreeRTOS, “Direct-to-Task Notifications,” FreeRTOS Documentation. [Online]. Available: https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/03-Direct-to-task-notifications/01-Task-notifications

[9] FreeRTOS, “xTaskNotifyGive() and xTaskNotifyGiveIndexed(),” FreeRTOS Documentation. [Online]. Available: https://freertos.org/xTaskNotifyGive.html

[10] FreeRTOS, “xTaskNotifyWait() / ulTaskNotifyTake() API Reference,” FreeRTOS Documentation. [Online]. Available: https://freertos.org/xTaskNotifyWait.html

Appendix A

Full Code

Algorithm 1: Solar Tracking Code via Photoresistor Matrix in FreeRTOS

```
#include <Arduino_FreeRTOS.h>
#include <task.h>
#include <math.h>

// ==========================
// Hardware pin assignment
// ==========================
#define MOTOR_IN1 11
#define MOTOR_IN2 12
#define ENCA 3
#define ENCB 2

// Four LDRs arranged as:
// A0 A1
// A2 A3
const int ldrPins[] = {A0, A1, A2, A3};
const int numSensors = 4;

// Encoder count at output shaft
volatile long encoderCount = 0;

// ==========================
// Shared RTOS control state
// ==========================
volatile float targetAngleDeg = 0.0f;
volatile float currentAngleDegShared = 0.0f;
volatile float controlEffortShared = 0.0f;
volatile int motorDirectionShared = 0;
volatile int motorPwmShared = 0;

// Shared sensor/debug variables for telemetry
volatile int A0_raw_shared = 0;
volatile int A1_raw_shared = 0;
volatile int A2_raw_shared = 0;
volatile int A3_raw_shared = 0;

volatile int A0_score_shared = 0;
volatile int A1_score_shared = 0;
volatile int A2_score_shared = 0;
volatile int A3_score_shared = 0;

volatile int topScore_shared = 0;
volatile int bottomScore_shared = 0;
volatile int leftScore_shared = 0;
volatile int rightScore_shared = 0;
```

```
volatile float xPos_shared = 0.0f;
volatile float yPos_shared = 0.0f;
volatile float verticalBias_shared = 0.0f;

// ==========================
// System constants
// ==========================
const float COUNTS_PER_REV = 120.0f;
const float MAX_SAFE_ANGLE = 48.0f;
const float ANGLE_TOL = 3.0f;

// ==========================
// LDR sector logic settings
// ==========================
const bool LDR_BRIGHTER_IS_LOWER = true;
const float CENTER_BIAS_THRESHOLD = 0.12f;
const unsigned long SECTOR_CONFIRM_MS = 400;

// ==========================
// Discrete target angles
// ==========================
const float ANGLE_POS = 45.0f;
const float ANGLE_ZERO = 0.0f;
const float ANGLE_NEG = -45.0f;

// Direction inversion flags
const bool INVERT_MOTOR = true;
const bool INVERT_ENCODER = false;

// ==========================
// PD controller gains
// ==========================
const float Kp_pos = 3.0f;
const float Kd_pos = 1.8f;

// ==========================
// Motor command shaping
// ==========================
const int PWM_MAX = 170;
const int PWM_MIN_MOVING = 55;
const float ANGLE_DEADBAND = 1.0f;

// Sensor task period
const TickType_t SENSOR_PERIOD_TICKS = pdMS_TO_TICKS(50);

// ==========================
// Sector classification
// ==========================
enum Sector {
  SECTOR_CENTER,
  SECTOR_TOP,
  SECTOR_BOTTOM
};
```

```
Sector currentSector = SECTOR_CENTER;
Sector candidateSector = SECTOR_CENTER;
unsigned long candidateStartTime = 0;

// ==========================
// Transition handling
// ==========================
enum TransitionState {
  TRANSITION_IDLE,
  TRANSITION_TO_CENTER_THEN_FINAL
};

volatile TransitionState transitionState = TRANSITION_IDLE;
volatile float pendingFinalTarget = 0.0f;

// ==========================
// Task handles for notification chain
// ==========================
TaskHandle_t sensorTaskHandle = NULL;
TaskHandle_t controlTaskHandle = NULL;
TaskHandle_t driverTaskHandle = NULL;
TaskHandle_t telemetryTaskHandle = NULL;

// Task prototypes
void vSensorTask(void *pvParameters);
void vControlTask(void *pvParameters);
void vDriverTask(void *pvParameters);
void vTelemetryTask(void *pvParameters);

// Encoder ISR prototype
void readEncoder();

// ================================================================
// Convert encoder counts to output angle in degrees
// ================================================================
float countsToDegrees(long counts) {
  return (counts * 360.0f) / COUNTS_PER_REV;
}

// ================================================================
// Read current angle atomically from encoder count
// ================================================================
float getCurrentAngle() {
  noInterrupts();
  long count = encoderCount;
  interrupts();
  return countsToDegrees(count);
}

// ================================================================
// Disable motor drive
// ================================================================
void stopMotor() {
```

```
  digitalWrite(MOTOR_IN1, LOW);
  digitalWrite(MOTOR_IN2, LOW);
}

// ============================================================
// Apply raw motor command
// direction: +1 or -1 in control-space sign convention
// pwm: 0 to 255
// ============================================================
void driveMotorRaw(int pwm, int direction) {
  pwm = constrain(pwm, 0, 255);

  if (INVERT_MOTOR) {
    direction = -direction;
  }

  if (direction > 0) {
    analogWrite(MOTOR_IN1, pwm);
    digitalWrite(MOTOR_IN2, LOW);
  } else if (direction < 0) {
    digitalWrite(MOTOR_IN1, LOW);
    analogWrite(MOTOR_IN2, pwm);
  } else {
    stopMotor();
  }
}

// ============================================================
// Convert raw ADC value into a brightness score
// ============================================================
int sensorToLightScore(int rawValue) {
  if (LDR_BRIGHTER_IS_LOWER) {
    return 1023 - rawValue;
  } else {
    return rawValue;
  }
}

// ============================================================
// Convert sector enum to readable label
// ============================================================
const char* sectorName(Sector s) {
  if (s == SECTOR_TOP) return "TOP";
  if (s == SECTOR_BOTTOM) return "BOTTOM";
  return "CENTER";
}

// ============================================================
// Map sector to commanded panel angle
// ============================================================
float sectorToAngle(Sector s) {
  if (s == SECTOR_TOP) return ANGLE_POS;
  if (s == SECTOR_BOTTOM) return ANGLE_NEG;
  return ANGLE_ZERO;
```

```
}

// ============================================================
// Classify LDR readings into TOP, CENTER, or BOTTOM sector
// ============================================================
Sector classifySector(
  int A0_raw, int A1_raw, int A2_raw, int A3_raw,
  int &A0_score, int &A1_score, int &A2_score, int &A3_score,
  int &topScore, int &bottomScore, int &leftScore, int &rightScore,
  float &xPos, float &yPos, float &verticalBias
) {
  A0_score = sensorToLightScore(A0_raw);
  A1_score = sensorToLightScore(A1_raw);
  A2_score = sensorToLightScore(A2_raw);
  A3_score = sensorToLightScore(A3_raw);

  topScore = A0_score + A1_score;
  bottomScore = A2_score + A3_score;
  leftScore = A0_score + A2_score;
  rightScore = A1_score + A3_score;

  int totalScore = A0_score + A1_score + A2_score + A3_score;

  xPos = 0.0f;
  yPos = 0.0f;
  verticalBias = 0.0f;

  if (totalScore > 0) {
    xPos = ((rightScore - leftScore) * 100.0f) / totalScore;
    yPos = ((topScore - bottomScore) * 100.0f) / totalScore;
    verticalBias = (float)(topScore - bottomScore) / (float)totalScore;
  }

  if (verticalBias > CENTER_BIAS_THRESHOLD) {
    return SECTOR_TOP;
  } else if (verticalBias < -CENTER_BIAS_THRESHOLD) {
    return SECTOR_BOTTOM;
  } else {
    return SECTOR_CENTER;
  }
}

// ============================================================
// Encoder ISR
// ENCA rising edge is used as the count event.
// ENCB determines the sign of rotation.
// ============================================================
void readEncoder() {
  int b = digitalRead(ENCB);
  int dir = (b == HIGH) ? 1 : -1;

  if (INVERT_ENCODER) {
    dir = -dir;
  }
```

```
  encoderCount += dir;
}

// ================================================================
// System setup
// ================================================================
void setup() {
  Serial.begin(115200);

  pinMode(MOTOR_IN1, OUTPUT);
  pinMode(MOTOR_IN2, OUTPUT);
  pinMode(ENCA, INPUT_PULLUP);
  pinMode(ENCB, INPUT_PULLUP);

  attachInterrupt(digitalPinToInterrupt(ENCA), readEncoder, RISING);

  stopMotor();

  noInterrupts();
  encoderCount = 0;
  interrupts();

  targetAngleDeg = 0.0f;
  currentSector = SECTOR_CENTER;
  candidateSector = SECTOR_CENTER;
  candidateStartTime = millis();
  transitionState = TRANSITION_IDLE;
  pendingFinalTarget = 0.0f;

  // Priority hierarchy:
  // Control highest, Sensor and Driver middle, Telemetry lowest
  xTaskCreate(vSensorTask, "Sensor", 384, NULL, 2, &sensorTaskHandle);
  xTaskCreate(vControlTask, "Control", 384, NULL, 3, &controlTaskHandle);
  xTaskCreate(vDriverTask, "Driver", 256, NULL, 2, &driverTaskHandle);
  xTaskCreate(vTelemetryTask, "Telemetry", 512, NULL, 1, &telemetryTaskHandle);
}

void loop() {
  // Unused because FreeRTOS tasks perform all runtime behavior.
}

// ================================================================
// Sensor Task
//
// Periodic source task. Reads LDRs, updates sector/target logic,
// then notifies the control task. This establishes a deterministic
// sensor-to-control handoff.
// ================================================================
void vSensorTask(void *pvParameters) {
  (void)pvParameters;

  TickType_t xLastWakeTime = xTaskGetTickCount();
```

```
for (;;) {
  int A0_raw = analogRead(ldrPins[0]);
  int A1_raw = analogRead(ldrPins[1]);
  int A2_raw = analogRead(ldrPins[2]);
  int A3_raw = analogRead(ldrPins[3]);

  int A0_score, A1_score, A2_score, A3_score;
  int topScore, bottomScore, leftScore, rightScore;
  float xPos, yPos, verticalBias;

  Sector detectedSector = classifySector(
    A0_raw, A1_raw, A2_raw, A3_raw,
    A0_score, A1_score, A2_score, A3_score,
    topScore, bottomScore, leftScore, rightScore,
    xPos, yPos, verticalBias
  );

  taskENTER_CRITICAL();
  A0_raw_shared = A0_raw;
  A1_raw_shared = A1_raw;
  A2_raw_shared = A2_raw;
  A3_raw_shared = A3_raw;

  A0_score_shared = A0_score;
  A1_score_shared = A1_score;
  A2_score_shared = A2_score;
  A3_score_shared = A3_score;

  topScore_shared = topScore;
  bottomScore_shared = bottomScore;
  leftScore_shared = leftScore;
  rightScore_shared = rightScore;

  xPos_shared = xPos;
  yPos_shared = yPos;
  verticalBias_shared = verticalBias;
  taskEXIT_CRITICAL();

  float detectedTarget = sectorToAngle(detectedSector);

  if (detectedSector != currentSector) {
    if (detectedSector != candidateSector) {
      candidateSector = detectedSector;
      candidateStartTime = millis();
    } else {
      if (millis() - candidateStartTime >= SECTOR_CONFIRM_MS) {
        float oldTarget;
        taskENTER_CRITICAL();
        oldTarget = targetAngleDeg;
        taskEXIT_CRITICAL();

        float newTarget = detectedTarget;

        // Force center pass for direct +45 <-> -45 transitions
```

```
            if ((oldTarget > 1.0f && newTarget < -1.0f) || (oldTarget < -1.0f && newTarget >
1.0f)) {
              taskENTER_CRITICAL();
              targetAngleDeg = ANGLE_ZERO;
              pendingFinalTarget = newTarget;
              transitionState = TRANSITION_TO_CENTER_THEN_FINAL;
              taskEXIT_CRITICAL();
            } else {
              taskENTER_CRITICAL();
              targetAngleDeg = newTarget;
              transitionState = TRANSITION_IDLE;
              pendingFinalTarget = 0.0f;
              taskEXIT_CRITICAL();
            }

            currentSector = detectedSector;
          }
        }
      } else {
        candidateSector = currentSector;
        candidateStartTime = millis();
      }

      // Deterministic release of control task after fresh sensor update
      xTaskNotifyGive(controlTaskHandle);

      vTaskDelayUntil(&xLastWakeTime, SENSOR_PERIOD_TICKS);
    }
}

// ============================================================
// Control Task
//
// Waits for sensor notification, then computes one PD update.
// After producing a new motor command, it notifies the driver task.
// This makes control execution event-driven by sensor completion.
// ============================================================
void vControlTask(void *pvParameters) {
  (void)pvParameters;

  float lastError = 0.0f;
  TickType_t lastWakeTick = xTaskGetTickCount();

  for (;;) {
    // Block until sensor task provides a new update
    ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

    TickType_t nowTick = xTaskGetTickCount();
    float dt = (float)(nowTick - lastWakeTick) * ((float)portTICK_PERIOD_MS / 1000.0f);
    if (dt <= 0.0f) {
      dt = 0.02f;
    }
    lastWakeTick = nowTick;
```

```
    float currentAngle = getCurrentAngle();

    float localTarget;
    TransitionState localTransitionState;
    float localPendingFinalTarget;

    taskENTER_CRITICAL();
    currentAngleDegShared = currentAngle;
    localTarget = targetAngleDeg;
    localTransitionState = transitionState;
    localPendingFinalTarget = pendingFinalTarget;
    taskEXIT_CRITICAL();

    // Complete center pass once panel is near zero
    if (localTransitionState == TRANSITION_TO_CENTER_THEN_FINAL) {
      if (fabs(currentAngle - ANGLE_ZERO) <= ANGLE_TOL) {
        taskENTER_CRITICAL();
        targetAngleDeg = localPendingFinalTarget;
        transitionState = TRANSITION_IDLE;
        pendingFinalTarget = 0.0f;
        localTarget = targetAngleDeg;
        taskEXIT_CRITICAL();
      }
    }

    float error = localTarget - currentAngle;
    float dError = (error - lastError) / dt;
    lastError = error;

    if (fabs(currentAngle) > MAX_SAFE_ANGLE) {
      taskENTER_CRITICAL();
      motorDirectionShared = 0;
      motorPwmShared = 0;
      controlEffortShared = 0.0f;
      taskEXIT_CRITICAL();

      xTaskNotifyGive(driverTaskHandle);
      continue;
    }

    float u = Kp_pos * error + Kd_pos * dError;

    int cmdDir = 0;
    int pwmCmd = 0;

    if (fabs(error) <= ANGLE_DEADBAND) {
      cmdDir = 0;
      pwmCmd = 0;
      u = 0.0f;
    } else {
      cmdDir = (u > 0.0f) ? 1 : -1;
      pwmCmd = (int)fabs(u);

      if (pwmCmd > 0 && pwmCmd < PWM_MIN_MOVING) {
```

```
        pwmCmd = PWM_MIN_MOVING;
      }
      if (pwmCmd > PWM_MAX) {
        pwmCmd = PWM_MAX;
      }
    }

    taskENTER_CRITICAL();
    motorDirectionShared = cmdDir;
    motorPwmShared = pwmCmd;
    controlEffortShared = u;
    taskEXIT_CRITICAL();

    // Deterministic handoff of computed command to driver
    xTaskNotifyGive(driverTaskHandle);
  }
}

// ============================================================
// Driver Task
//
// Waits for control notification, then applies exactly one motor
// command update. This isolates hardware actuation from control
// computation and makes the command path explicit.
// ============================================================
void vDriverTask(void *pvParameters) {
  (void)pvParameters;

  for (;;) {
    // Block until control task publishes a fresh command
    ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

    int dir;
    int pwm;

    taskENTER_CRITICAL();
    dir = motorDirectionShared;
    pwm = motorPwmShared;
    taskEXIT_CRITICAL();

    if (dir == 0 || pwm == 0) {
      stopMotor();
    } else {
      driveMotorRaw(pwm, dir);
    }
  }
}

// ============================================================
// Telemetry Task
//
// Periodic low-priority debug output. This task is intentionally
// decoupled from the deterministic sensor-control-driver chain.
// ============================================================
```

```
void vTelemetryTask(void *pvParameters) {
  (void)pvParameters;

  TickType_t xLastWakeTime = xTaskGetTickCount();
  const TickType_t xFrequency = pdMS_TO_TICKS(200);

  for (;;) {
    float currentAngle, targetAngle, controlEffort;
    int motorDir, motorPwm;

    int A0_raw, A1_raw, A2_raw, A3_raw;
    int A0_score, A1_score, A2_score, A3_score;
    int topScore, bottomScore, leftScore, rightScore;
    float xPos, yPos, verticalBias;

    taskENTER_CRITICAL();
    currentAngle = currentAngleDegShared;
    targetAngle = targetAngleDeg;
    controlEffort = controlEffortShared;
    motorDir = motorDirectionShared;
    motorPwm = motorPwmShared;

    A0_raw = A0_raw_shared;
    A1_raw = A1_raw_shared;
    A2_raw = A2_raw_shared;
    A3_raw = A3_raw_shared;

    A0_score = A0_score_shared;
    A1_score = A1_score_shared;
    A2_score = A2_score_shared;
    A3_score = A3_score_shared;

    topScore = topScore_shared;
    bottomScore = bottomScore_shared;
    leftScore = leftScore_shared;
    rightScore = rightScore_shared;

    xPos = xPos_shared;
    yPos = yPos_shared;
    verticalBias = verticalBias_shared;
    taskEXIT_CRITICAL();

    Serial.print("Target:");
    Serial.print(targetAngle, 1);
    Serial.print("\tAngle:");
    Serial.print(currentAngle, 1);
    Serial.print("\tErr:");
    Serial.print(targetAngle - currentAngle, 1);
    Serial.print("\tU:");
    Serial.print(controlEffort, 1);
    Serial.print("\tDir:");
    Serial.print(motorDir);
    Serial.print("\tPWM:");
    Serial.print(motorPwm);
```

```
    Serial.print("\tRAW A0:");
    Serial.print(A0_raw);
    Serial.print("\tA1:");
    Serial.print(A1_raw);
    Serial.print("\tA2:");
    Serial.print(A2_raw);
    Serial.print("\tA3:");
    Serial.print(A3_raw);

    Serial.print("\tS A0:");
    Serial.print(A0_score);
    Serial.print("\tA1:");
    Serial.print(A1_score);
    Serial.print("\tA2:");
    Serial.print(A2_score);
    Serial.print("\tA3:");
    Serial.print(A3_score);

    Serial.print("\tTop:");
    Serial.print(topScore);
    Serial.print("\tBottom:");
    Serial.print(bottomScore);
    Serial.print("\tLeft:");
    Serial.print(leftScore);
    Serial.print("\tRight:");
    Serial.print(rightScore);

    Serial.print("\tX:");
    Serial.print(xPos, 1);
    Serial.print("\tY:");
    Serial.print(yPos, 1);
    Serial.print("\tVBias:");
    Serial.println(verticalBias, 3);

    vTaskDelayUntil(&xLastWakeTime, xFrequency);
  }
}
```

Appendix B: Video Demo

[1] C. Lowden, "Initial Testing - 462 Solar Panel Tracker," *YouTube Shorts*, Apr. 17, 2026. Accessed: Apr. 27, 2026. [Online]. Available: https://youtube.com/shorts/2tpek7yidHo

[2] C. Lowden, "Final Test - 462 Panel Tracker" *YouTube Shorts*, Apr. 27, 2026. Accessed: Apr. 27, 2026. [Online]. Available: https://youtube.com/shorts/6QvVgAxkgGk